Week 6, Lesson 6

Database Services in AWS

Chapter 4 – Amazon Database Services (Song, 2023, pp. 79–104)

Outline

1. Relational vs. NoSQL vs. in-memory databases
2. Amazon RDS, DynamoDB, and ElastiCache
3. Amazon Redshift (data warehouse service)

Dr. Rohit Kumar Kasera
Lecturer

Relational vs. NoSQL vs. in-memory databases

Aspect	Relational Databases (SQL)	NoSQL Databases	In-Memory Databases
Definition	Store data in structured tables with relationships	Store non-tabular or flexible data formats	Store data primarily in RAM for very fast access
Examples	PostgreSQL, MySQL, Oracle Database	MongoDB, Cassandra, DynamoDB	Redis, Memcached, SAP HANA
Data Model	Tables, rows, columns	Document, key-value, graph, column-family	Mostly key-value
Schema	Fixed/predefined	Flexible or schema-less	Flexible
Query Language	SQL	Varies by database	Simple commands/APIs
Relationships	Strong support using joins	Limited or embedded relationships	Minimal relationship support
Consistency	Strong ACID compliance	Often eventual consistency	Varies
Scalability	Vertical scaling mainly	Horizontal scaling	Horizontal scaling

Aspect	Relational Databases (SQL)	NoSQL Databases	In-Memory Databases
Performance	Good for complex queries	Good for large distributed systems	Extremely fast
Storage Location	Disk-based	Disk/distributed storage	RAM (memory)
Latency	Moderate	Fast	Very low (microseconds/ms)
Best Use Cases	Banking, ERP, transactions	Social media, IoT, big data	Caching, sessions, leaderboards
Advantages	Reliable, structured, mature ecosystem	Flexible, scalable, handles huge data	Ultra-fast access speed
Disadvantages	Harder to scale, rigid schema	Less standardized, weaker joins	Expensive memory, possible volatility
Typical Example Scenario	Online banking system	Real-time recommendation engine	Website cache/session store

Database Type Comparison

Relative strengths of relational, NoSQL, and in-memory databases across scalability, flexibility, consistency, and speed.



AWS Database Service Comparison

EIU

Service	Type	Use Case	Scaling	Management
RDS	Relational	Traditional apps	Vertical	Managed
Aurora	Relational	High performance	Horizontal reads	Fully managed
DynamoDB	NoSQL	Web scale apps	Horizontal	Serverless
ElastiCache	In-Memory	Caching layer	Horizontal	Managed
Redshift	Data warehouse	Analytics/BI	Horizontal	Managed

Each service is optimized for distinct workload patterns. Understanding these differences is critical for architectural decisions.

Amazon RDS, DynamoDB, and ElastiCache

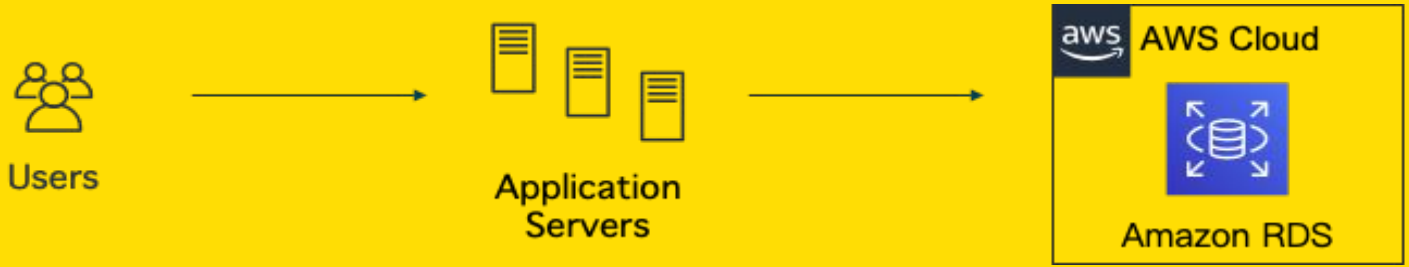
Amazon RDS

Amazon RDS (Relational Database Service) is a managed database service that simplifies setup, operation, and scaling of relational databases in the cloud. AWS handles routine tasks like provisioning, patching, backup, recovery, and failure detection.
Amazon Aurora (MySQL and PostgreSQL compatible)

Supported Engines:

- MySQL
- PostgreSQL
- MariaDB
- Oracle database
- Microsoft SQL Server

Key Benefits: Automated backups, software patching, automatic failure detection, and recovery. Multi-AZ deployment provides high availability and automatic failover to standby instances.

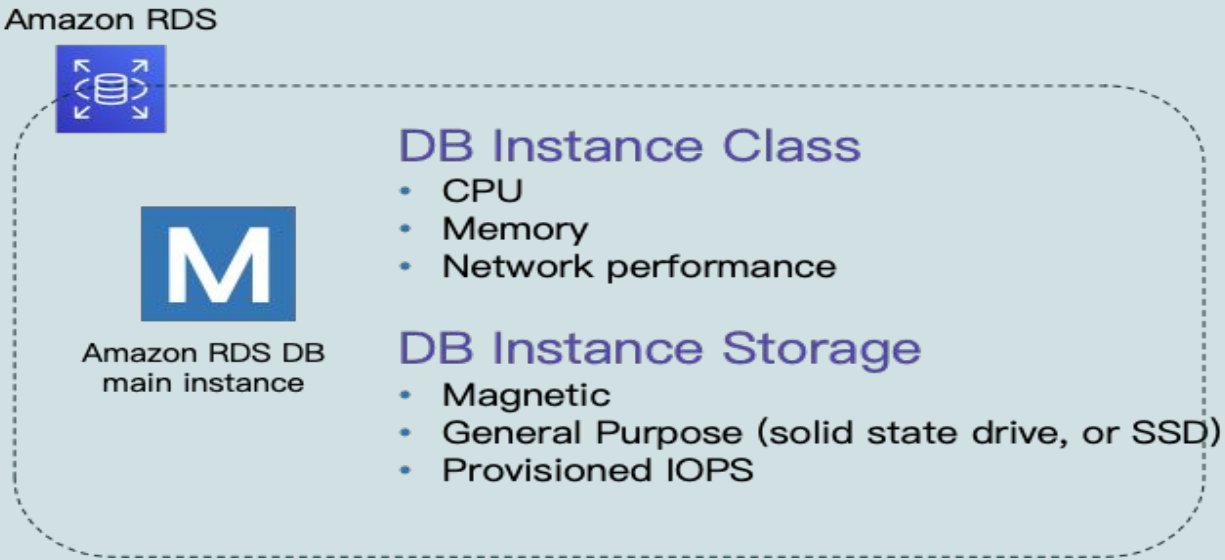


RDS Architecture

Amazon RDS is built on a distributed architecture designed to provide scalability, availability, and reliability for your database workloads.
Core components:

- Database instances
- Multi-Availability Zone deployments
- Storage
- Network integration

DB Instances

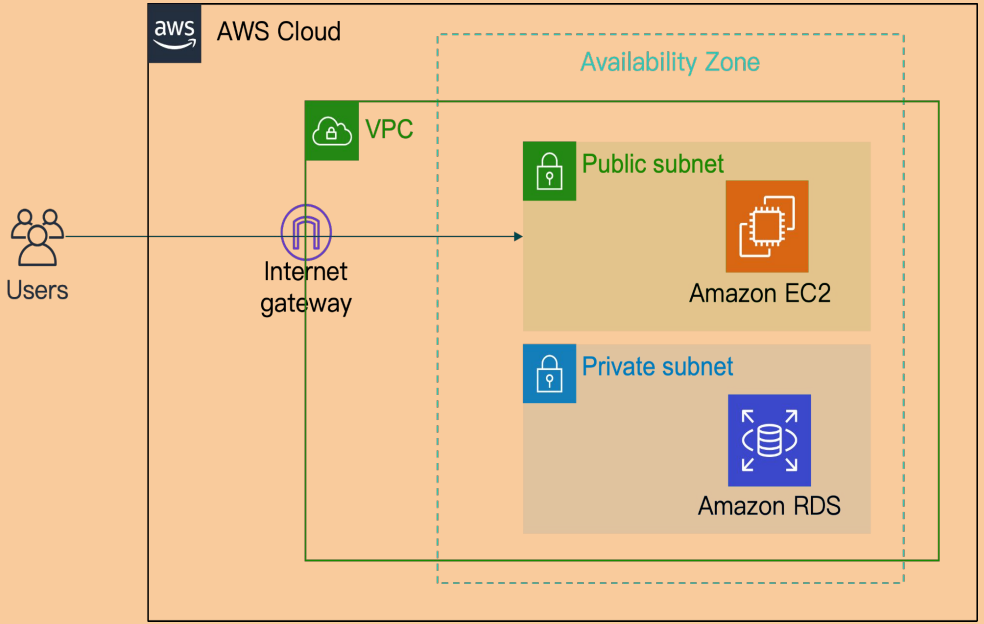


Network integration

RDS runs inside:

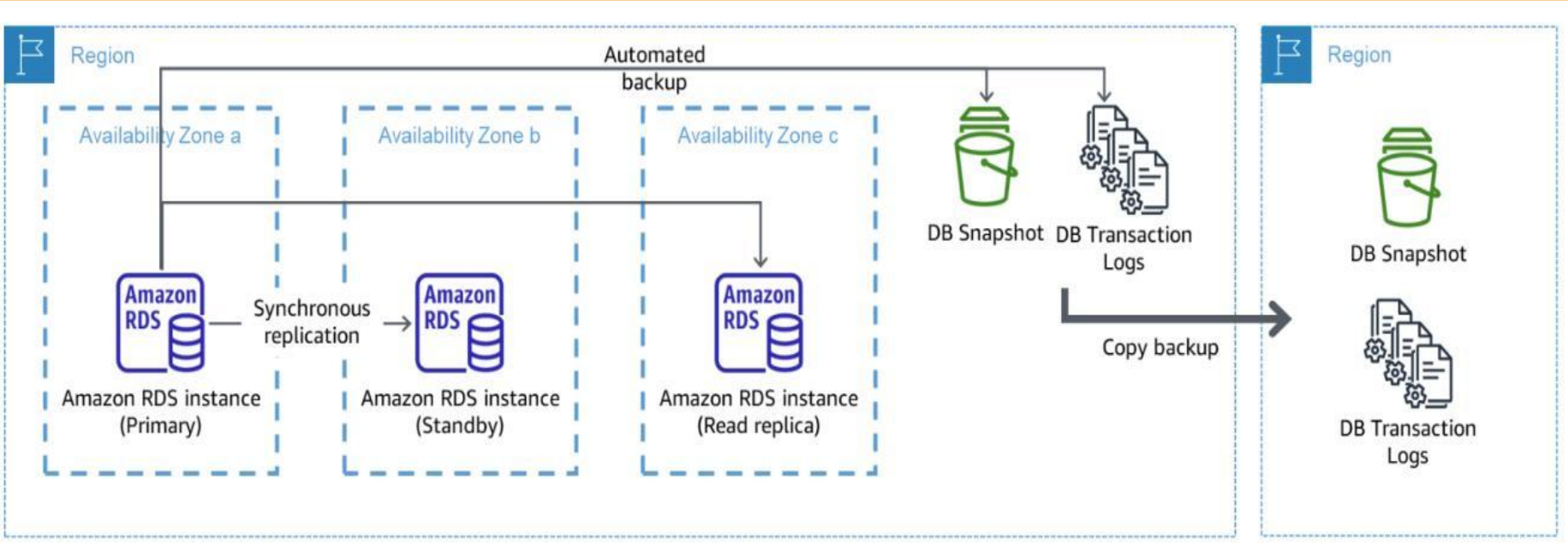
- VPC
- Subnet Group
- Security Groups

You can launch RDS instances within a Virtual Private Cloud (VPC), which lets you define network configurations and isolate your database in a secure environment. VPC integration also provides secure communication with other AWS resources like EC2 instances, Lambda functions, and S3 buckets.



Multi-Availability Zone deployments

- For high availability, RDS can deploy a database across multiple Availability Zones (AZs). RDS replicates the primary database instance in one AZ to a standby instance in another AZ.
- In the event of failure, RDS automatically switches to the standby instance, which ensures minimal downtime.



Multi-Availability Zone deployments

Features:

- Automatic failover
- Synchronous replication
- Managed disaster recovery
- Minimal downtime

Used for:

- Production systems
- Critical workloads

Features

- Offers asynchronous replication
- Can be promoted to primary if needed

Functionality

- Use for read-heavy database workloads
- Offload read queries

Use cases

Web and mobile applications	High throughput Massive storage scalability High availability
Ecommerce applications	Low-cost database Data security Fully managed solution
Mobile and online games	Rapidly grow capacity Automatic scaling Database monitoring

AWS RDS Storage

Provisioned storage –

- No charge
  - Backup storage of up to 100 percent of database storage for an active database
- Charge (*GB/month*)
  - Backup storage for terminated DB instances

Additional storage –

- Charge (*GB/month*)
  - Backup storage in addition to provisioned storage

Amazon RDS: Deployment type and data transfer

Requests –

- The number of input and output requests that are made to the database

Deployment type—Storage and I/O charges vary, depending on whether you deploy to –

- Single Availability Zone
- Multiple Availability Zones

Data transfer –

- No charge for inbound data transfer
- Tiered charges for outbound data transfer

Understanding Amazon DynamoDB

- Fast and flexible NoSQL database service for any scale.
  - NoSQL database tables
  - Virtually unlimited storage
  - Items can have differing attributes
  - Low-latency queries
  - Scalable read / write throughput

Core Architecture

- **Table:** A collection of data. (Equivalent to a Table in SQL).
- **Item:** A single record in the table. (Equivalent to a Row in SQL).
- **Attribute:** A single data element in an item. (Equivalent to a Column in SQL).

- The most critical design decision in DynamoDB is your Primary Key.
- It determines how data is stored and queried

Partition Key (Simple Primary Key):

- Composed of one attribute (e.g., UserID).
- DynamoDB uses this key to hash and distribute data across physical partitions.
- **Constraint:** You can only query for an exact match on this key (e.g., GetItem where UserID = 123).

Partition Key + Sort Key (Composite Primary Key):

- Composed of two attributes (e.g., Artist as Partition Key and SongTitle as Sort Key).
- All items with the same Partition Key are stored together physically and sorted by the Sort Key.
- **Constraint:** Uniqueness is enforced on the combination (Artist + SongTitle).
- **Power:** This allows rich queries, like "Get all songs by *Taylor Swift* (Partition) that start with the letter *L* (Sort)".

Use cases

- Develop software applications
- Create media metadata stores
- Deliver seamless retail experiences
- Scale gaming platforms
- Achieve highest level of application resilience

Creating a DynamoDB through AWS Cloudshell

```
aws dynamodb create-table \
--table-name Music \
--attributedefinitions \
AttributeName=Artist, AttributeType=S \
AttributeName=SongTitle, AttributeType=S \
--key-schema \
AttributeName=Artist,KeyType=HASH \
AttributeName=SongTitle, KeyType=RANGE \
--provisionedthroughput \
ReadCapacityUnits=5,WriteCapacityUnits=5 \
--table-class STANDARD
```

Add items to the DynamoDB table:

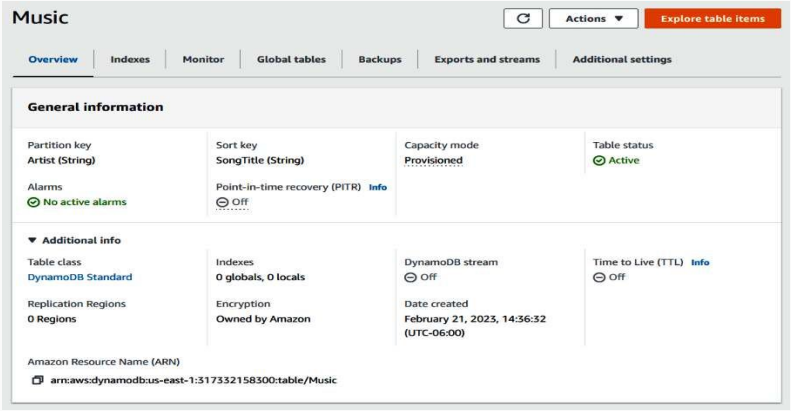
```
aws dynamodb put-item --table-name Music --item \
'{"Artist": {"S": "Broadway"}, "SongTitle": {"S": "Call Me"},
"AlbumTitle": {"S": "Somewhat Famous"}}'

aws dynamodb put-item --table-name Music --item \
'{"Artist": {"S": "Broadway"}, "SongTitle": {"S": "Howdy"},
"AlbumTitle": {"S": "Famous"}, "Awards": {"N": "2"}}'

aws dynamodb put-item --table-name Music --item \
'{"Artist": {"S": "Acme Band"}, "SongTitle": {"S": "Happy"},
"AlbumTitle": {"S": "About Life"}, "Awards": {"N": "10"}}'
```

Query the table in the AWS console

- Going to AWS Console | DynamoDB | Tables



Amazon ElastiCache

- ElastiCache is a fully managed, in-memory data caching service provided by Amazon in the cloud.
- ElastiCache supports two popular in-memory data stores, Redis and Memcached, which can be used as caching layers for a wide range of applications, including web applications, mobile applications, and gaming applications.
- ElastiCache removes undifferentiated heavy lifting of caching administrative tasks such as hardware provisioning, software patching, monitoring, backup and recovery.
- It also integrates with other AWS services, such as Amazon EC2, AWS Lambda, and Amazon RDS, for seamless integration with other parts of an AWS application stack.

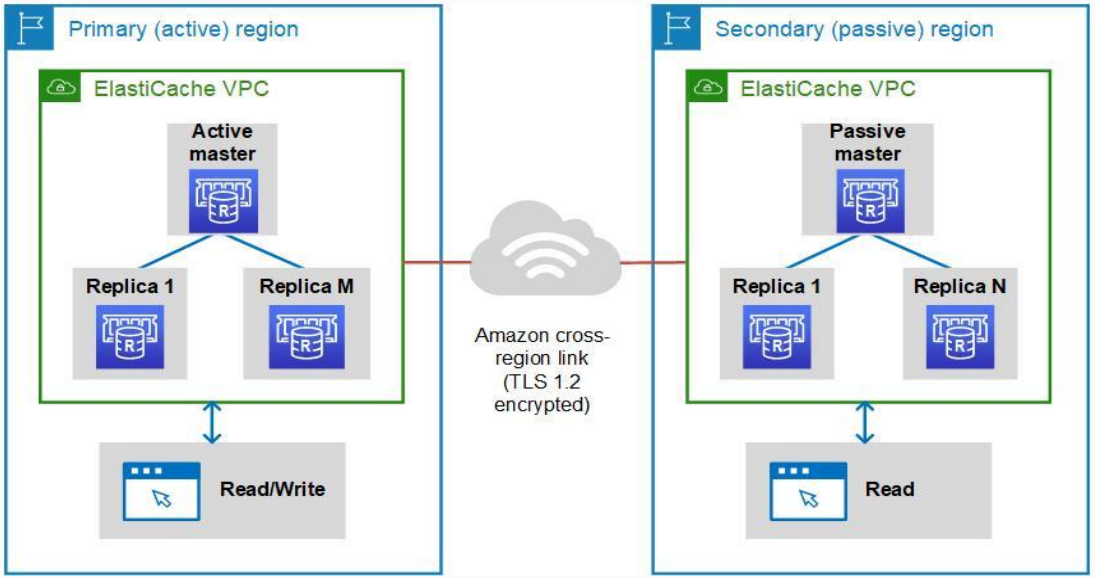
Monitoring & Observability

Integrated with:

- Amazon CloudWatch
- Enhanced metrics
- Event notifications

Important metrics:

- CPU usage
- Memory usage
- Evictions
- Cache hits/misses
- Replication lag
- Network throughput



Data Persistence

Redis supports persistence.

Options:

- RDB snapshots
- AOF (Append Only File)

Persistence stored in:

- Amazon S3 backups

Use persistence for:

- Session recovery
- Durable cache
- Fast restart

Feature	ElastiCache	RDS
Storage	In-memory	Disk-based
Speed	Microseconds	Milliseconds
Persistence	Optional	Persistent
Query Type	Key-value	SQL
Use Case	Caching	Primary database

Steps to create an ElastiCache instance and connect it to a client application:

1. **Create an EC2 instance with an SG**: "It is to allow default SSH from the computer where you will access the ElastiCache cluster.
   a. Use the VPC/subnet for the EC2 instance, which we will name EC2-1, with an SG of allowing SSH from my computer so we can access the EC2 instance from our home computer.
   b. Save the key pair and the instance's public IP address.
   c. Create an SG named **RedisSG** to allow SSH from the previous SG.
2. **Go to the Amazon ElastiCache service**: Click **Get started**, then click **Create cluster**.
   a. Choose **Create Redis cluster**.
   b. Choose **RedisSG** as the security group, and select **minimum con" guration**.

3. Access the Redis cluster from RedisInsight: Just like the PUTTY tool we used to access Linux EC2 instances in the cloud, RedisInsight is a tool to access the Redis cluster remotely.

- After launching RedisInsight, click Add Database, then fill in the information for the cluster.
- Fill in Host using the primary endpoint copied from the Redis cluster details, leaving the default Redis port of 6379.
- Since we will use the EC2 instance as the SSH tunnel, check Use SSH Tunnel and enter the instance's public IP address for Host, leaving the SSH port 22.
- Check Private Key and copy the key from the kp.pem file.
- Once the Redis database is added, we can use **Browser**, **Workbench,** and the **Analysis** tools.

Features:

- High availability
- Read scaling
- Automatic failover
- Simpler management

Best for:

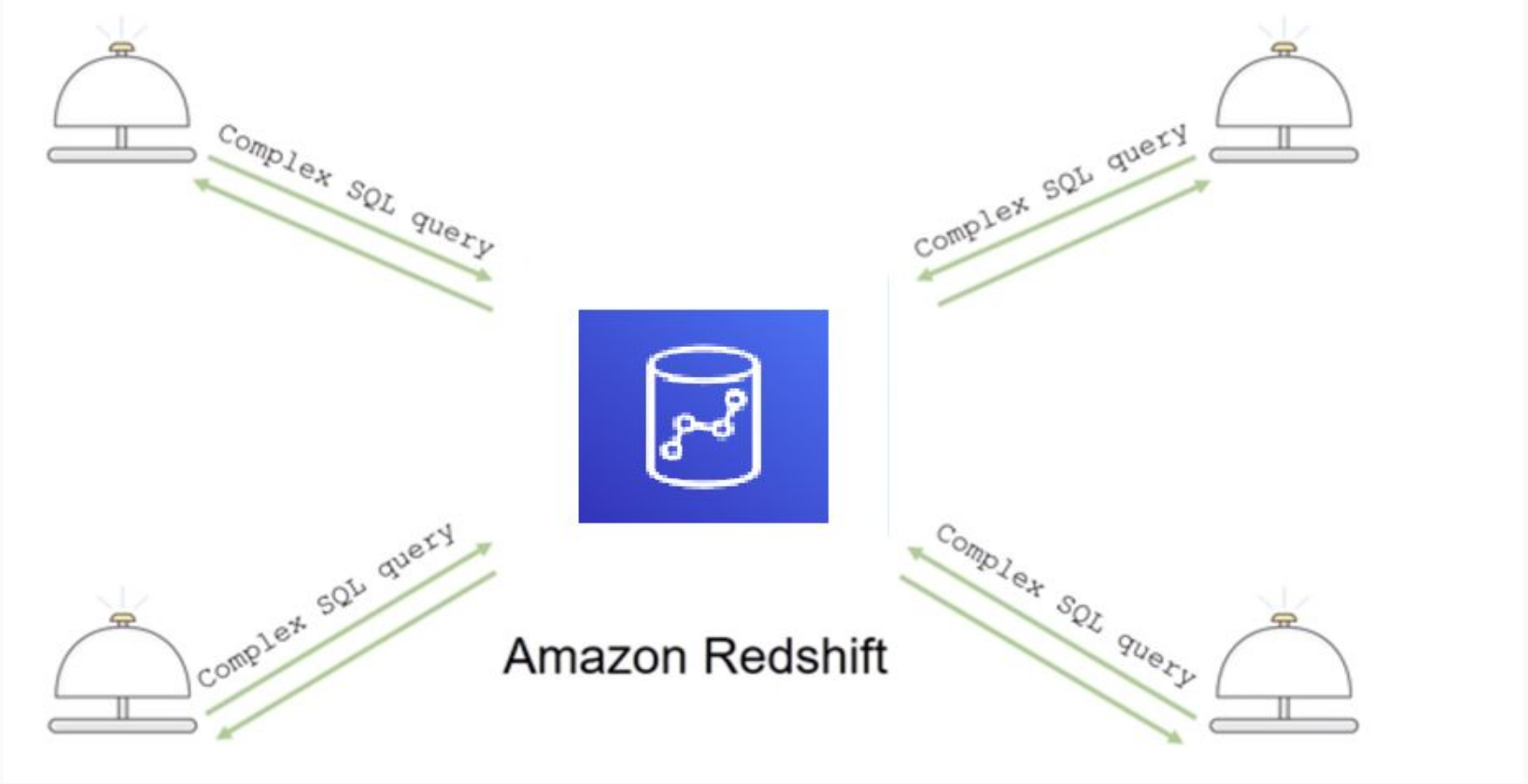
- Small-to-medium workloads
- Simpler caching needs

Amazon Redshift (data warehouse service)

- Amazon Redshift is a fast, fully managed data warehousing service in the cloud, enabling businesses to execute complex analytic queries on volumes of data.
- Thus minimizing delays and ensuring sound support for decision-making across organizations.
- Amazon Redshift is a flexible, massively scalable, cloud-based service that ranges from a few hundred gigabytes of data to several petabytes,
- It allows businesses to handle increasingly larger data sizes without much upfront investment.
- The architecture of Redshift is optimized for complex queries and analytics using techniques like columnar storage and massively parallel processing to deliver high-speed query performance.

Key Features of Amazon Redshift

- **Scalability:** Scale from a few hundreds of gigabytes to a petabyte or even more, allowing businesses to grow their data warehouses based on necessity.
  a. In its core is a columnar and MPP-based storage that ensures quick query performance, even over large datasets.
- **Integration:** Redshift seamlessly integrates with Amazon S3, Amazon RDS, AWS Glue, and much more to create a data ecosystem.
- **Cost-Effective:** Amazon Redshift is structured in a way such that it turns out to be cost-effective for you, with a couple of pricing options that enable one to pay just for storage and computing power.


Complex SQL query
Complex SQL query
Complex SQL query
Complex SQL query
Amazon Redshift

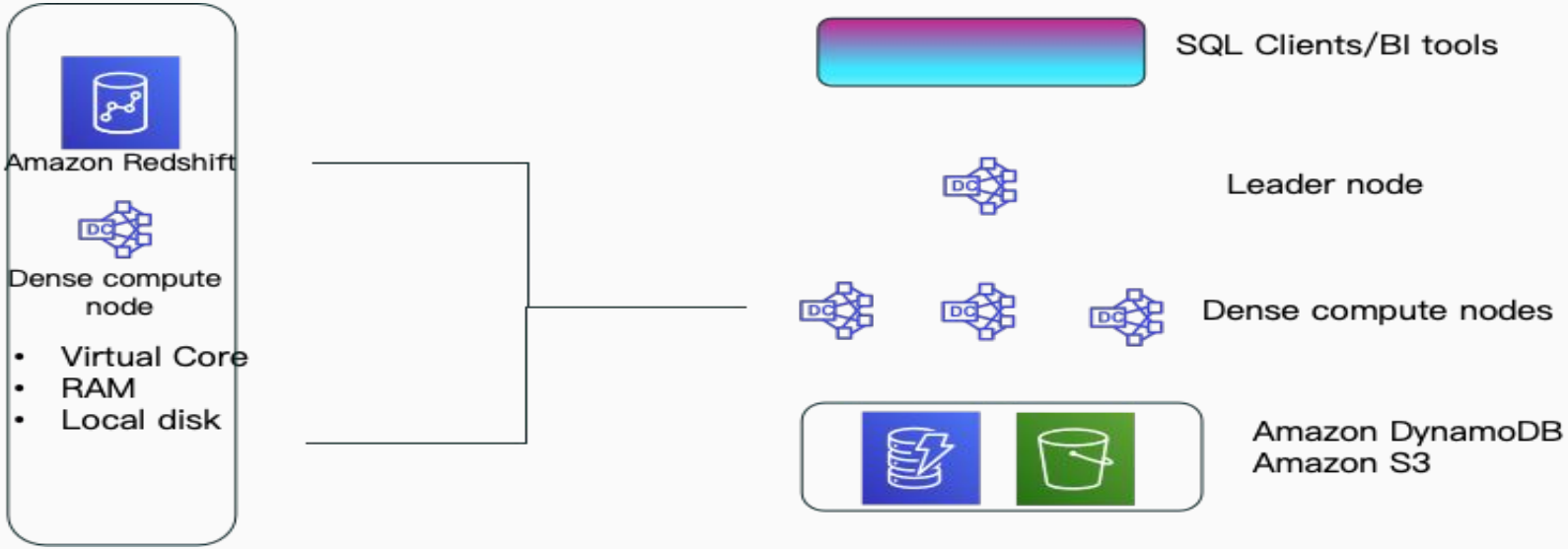
How Amazon Redshift Works?

- **Clusters and Nodes:** Redshift groups its resources into clusters.
  a. A cluster consists of one or more compute nodes. A leader node manages client connections and SQL processing.
  b. Compute nodes execute the queries and store data.
- **Data Storage:** Redshift organizes data in row format followed by organizing it columnar.
  a. This architecture minimizes the volume of disk reads and hence increases performance for analytical queries.
- **Query Execution:** Redshift runs each query in parallel on multiple nodes, enabling it to distribute workloads and process large data quantities with MPP architecture.

Use Cases:

- **Business Intelligence:** Companies have large datasets and use Redshift to process complex queries.
  a. Generate reports, and gain insights into their data for supporting decision-making processes.
- **Data Warehousing:** Primarily, Redshift provides a central data warehouse to store and analyze all data created in various sources.
- **Big Data Analytics:** Since it accommodates petabyte-scale data capacity.
  a. Redshift is large enough for an enterprise to analyze big data that allows them to observe any trends or patterns within their data.

Parallel processing architecture



- A computing architecture through which a single query workload or dataset is shared by multiple processors or nodes, where the pace of processing these queries is much quicker than one processor handling all these queries.
- Redshift adopts PP so it can allocate query processing to various compute nodes, which tremendously minimizes time whenever processing large datasets.

Further reading

- Chapter 4 – Amazon Database Services (Song, 2023, pp. 79–104)
- Amazon RDS. https://docs.aws.amazon.com/AmazonRDS/latest/gettingstartedguide/concepts.html
- AWS DynamoDB. https://aws.amazon.com/dynamodb/
- Amazon ElastiCache. https://aws.amazon.com/elasticache/
- Amazon Redshift https://www.geeksforgeeks.org/devops/amazon-redshift/
- AWS redshift. https://aws.amazon.com/redshift/

Thank you!

End